

AI Usage DISCLOSURE FORM

1. Team Details

Team Name: SRMIST_QueueQueueing

Project / Product Name: FixFlow — Smart Guided Troubleshooting Engine (Samsung PRISM GenAI Hackathon 2026, Theme 02)

Organization / Institution (if any): SRM Institute of Science and Technology

Submission Date: 30 September 2026

2. AI Usage Declaration

Did your team use any Artificial Intelligence (AI) in developing this project? Yes

Yes. (1) Claude Code (Anthropic, Claude Opus model, in the Claude desktop app) was our AI engineering assistant: planning, writing most of the code, tests, evaluation scripts, README and slide deck, under the team's direction, review and testing. (2) The product itself calls Google Gemini (Flash-Lite, Gemini API) at runtime for query enrichment and grounded step extraction; every LLM output is verified in code, with an offline rules fallback.

3. Purpose of AI Usage (Brief Details)

Idea generation / brainstorming Claude Code analysed the Theme 02 spec and rules with us and drafted the architecture and timeline; the team chose the theme, approach, scope and priorities.

Code generation or assistance Most of the Python engine, scripts and 48 automated tests were generated by Claude Code from our instructions, then reviewed, run and corrected with the team.

UI / UX design The demo UI (HTML/JS) was generated by Claude Code; layout, microphone handling and free-typed complaint support were changed after our own testing.

Content creation README, metrics report text, pitch script and the slide deck (built on the official template) were drafted by Claude Code; the team supplied details and reviewed them.

Data analysis AI-assisted scripts compared kit versions, built evaluation sets (paraphrases, held-out scenarios) and computed metrics; gold labels were drafted with AI assistance from the reference articles.

Testing / debugging Claude Code wrote and ran the test suite and fixed bugs it found (e.g. a wrong 'Screen lock' deeplink); the team tested the UI, Docker build and sample complaints manually.

Other Runtime AI inside the product: Google Gemini API (enrichment + grounded extraction) and the open-source local embedding model BAAI/bge-small-en-v1.5 for retrieval and the semantic cache.

4. Feature Origin Classification (Please add/delete based on the number of the feature)

1. Feature Name REST API and contract gate

2. Self-Generated/AI-Generated/Both Both

3. Description (include AI tools/platform Used, prompt used , Output Summary, Modification) Tools: Claude Code. Prompt: build a FastAPI service with POST /v1/troubleshoot and GET /health per the Theme 02 spec, enforcing every field rule in code and validating against the official schema.py. Output: app/main.py, app/validators.py and contract tests. Modification: the team checked spec compliance; the API was extended to accept siis_response as a string or {title, content} after reviewing the kit.

1. Feature Name Query enrichment and multi-complaint split

2. Self-Generated/AI-Generated/Both Both

3. Description (include AI tools/platform Used, prompt used , Output Summary, Modification) Tools: Claude Code (code), Gemini API (runtime). Prompt: normalise vague complaints, split numbered multi-complaint lines, generate 8-10 query variations in 5 registers. Output: app/enrich_rules.py and the enrichment prompt in app/llm_stages.py. Modification: device detection made brand-neutral after the official kit arrived; symptom patterns tuned during testing.

1. Feature Name Grounded step extraction (rules + LLM with citations)

2. Self-Generated/AI-Generated/Both Both

3. Description (include AI tools/platform Used, prompt used , Output Summary, Modification) Tools: Claude Code (code), Gemini API (runtime). Prompt: extract ordered, imperative steps only from the reference article; the LLM must cite numbered sentences and code must reject unsupported steps. Output: app/extract_rules.py, app/llm_stages.py. Modification: prompt tightened after reviewing plans that dropped sections or accepted irrelevant articles.

1. Feature Name Exact-screen deeplink mapping

2. Self-Generated/AI-Generated/Both Both

3. Description (include AI tools/platform Used, prompt used , Output Summary, Modification) Tools: Claude Code. Prompt: map each Settings step to the exact catalog entry using descriptions only (never the masked URI), with action- and depth-aware ranking. Output: app/catalog.py (Feature Graph, hybrid BM25 + embeddings). Modification: phrase check added after our gold labels exposed a wrong 'Screen lock' match; ablation confirmed 100% precision.

1. Feature Name Fast-path semantic cache

2. Self-Generated/AI-Generated/Both Both

3. Description (include AI tools/platform Used, prompt used , Output Summary, Modification) Tools: Claude Code. Prompt: serve repeat and paraphrased complaints in under 300 ms without the LLM, safely. Output: app/cache.py (exact + semantic L1, L2 on the normalised complaint, symptom guard, trust tiers). Modification: thresholds tuned on one paraphrase set and reported honestly on a held-out set.

1. Feature Name Evaluation harness, datasets and ablation

2. Self-Generated/AI-Generated/Both Both

3. Description (include AI tools/platform Used, prompt used , Output Summary, Modification) Tools: Claude Code. Prompt: measure every Appendix-C metric, build held-out paraphrase and 4-domain scenario sets, and compare mapping strategies. Output: eval/run_eval.py, eval/ablation.py, eval/datasets, metrics.md. Modification: the team reviewed the results; limitations (provider outage, one annotator) are reported openly.

1. Feature Name Demo UI

2. Self-Generated/AI-Generated/Both Both

3. Description (include AI tools/platform Used, prompt used , Output Summary, Modification) Tools: Claude Code. Prompt: a phone-style demo with ordered plan cards, Open/Verify via validation deeplinks, guided mode, voice input and an engine view. Output: ui/index.html, app/kb.py. Modification: after our testing, the layout was made phone-first, microphone errors are shown, and free-typed complaints are matched to knowledge-base articles.

1. Feature Name Presentation, README and pitch script

2. Self-Generated/AI-Generated/Both Both

3. Description (include AI tools/platform Used, prompt used , Output Summary, Modification) Tools: Claude Code (content), PowerPoint (export). Prompt: fill the official submission template section by section, write the README and a 4-presenter pitch script. Output: docs/SRMIST_QueueQueueing.pptx/.pdf, README.md, docs/PITCH.md. Modification: the team provided member details and the video, and reviewed the content.

...

5. Ethical & Compliance Confirmation

AI usage complies with guidelines and policies. **Yes**

No proprietary or copyrighted data misused. **I Agree**

6. Declaration & Sign-Off

Name of Team Representative: Abhinav Kumar (RA2411003010993, ak8045@srmist.edu.in)

Role: Primary member and team representative, Team SRMIST_QueueQueueing

Signature:

Date: 30 September 2026